AADS — Getting started with laboratories material

CLion is the official course environment

The official and supported IDE for the 2026/27 course edition is **JetBrains CLion 2026.2 or newer**, preferably the latest stable version available.

Download CLion from:

https://www.jetbrains.com/clion/download/

Students are eligible for a free JetBrains educational license through their university student status. The laboratory material is prepared and tested with CLion; other IDEs and custom development workflows are outside the supported course environment.

CLion normally configures a suitable compiler, debugger, and CMake toolchain. You are not expected to install or configure special versions of these tools.

Do not modify `CMakeLists.txt` unless an exercise explicitly asks you to do so.

Extract the complete laboratory

Laboratory material is distributed as a ZIP archive. Extract the complete archive to a normal directory before working on it.

Do not work directly inside the ZIP file and do not copy only the source files into another project. The archive also contains the build configuration, prepared Run configurations, example data, exercise descriptions, and tests.

A laboratory normally has this structure:

```
aads-lab-XX/
├── 01_first_exercise/
│   ├── README.md
│   ├── CMakeLists.txt
│   ├── .run/
│   ├── src/
│   ├── data/        # when input files are provided
│   └── tests/       # when automatic tests are provided
└── 02_second_exercise/
    ├── README.md
    ├── CMakeLists.txt
    ├── .run/
    └── src/
```

The README inside each exercise directory defines that exercise's problem, input, required behavior, output, constraints, supplied files, and examples. It is the authoritative specification of that exercise.

Open one exercise at a time

Every exercise is an independent project. Do **not** open the top-level `aads-lab-XX` directory as a single CLion project.

Open the directory of the exercise you want to work on. The correct directory is the one that directly contains `CMakeLists.txt`.

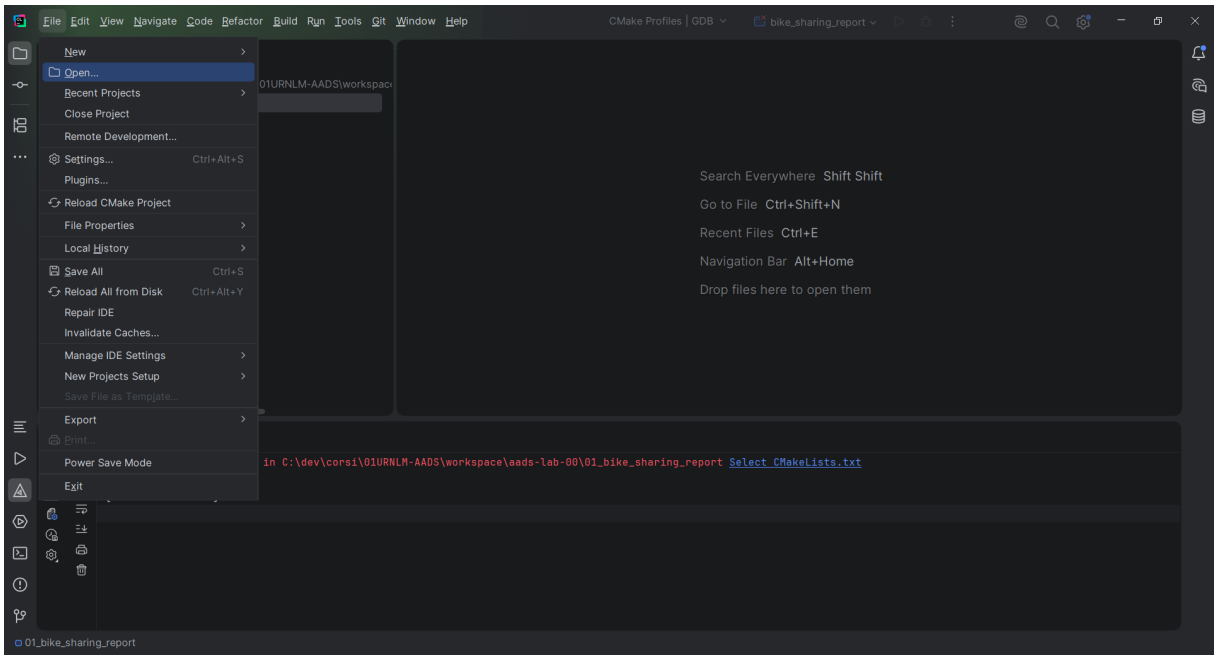
For example, given:

```
aads-lab-00/
├── 01_bike_sharing_report/
└── 02_monitoring_grid/
```

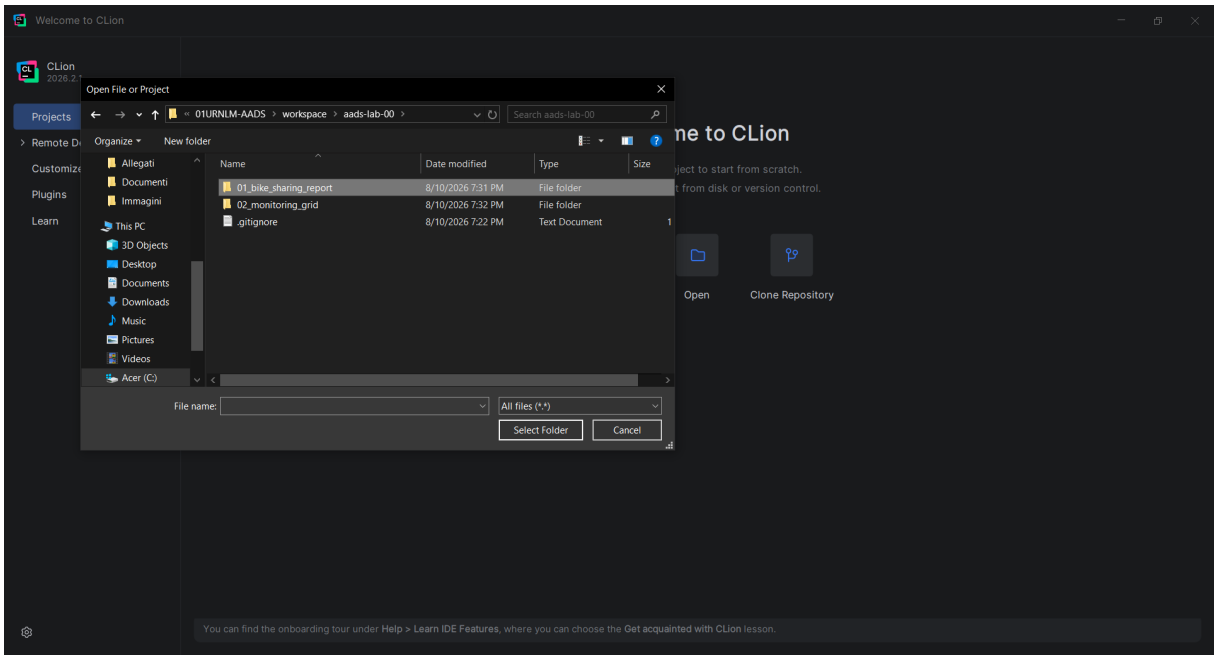
open `aads-lab-00/01_bike_sharing_report` for the first exercise. When you move to the second exercise, open `aads-lab-00/02_monitoring_grid` as a separate project.

Open and load the project in CLion

1. Start CLion and choose **Open**.



2. Select the individual exercise directory.

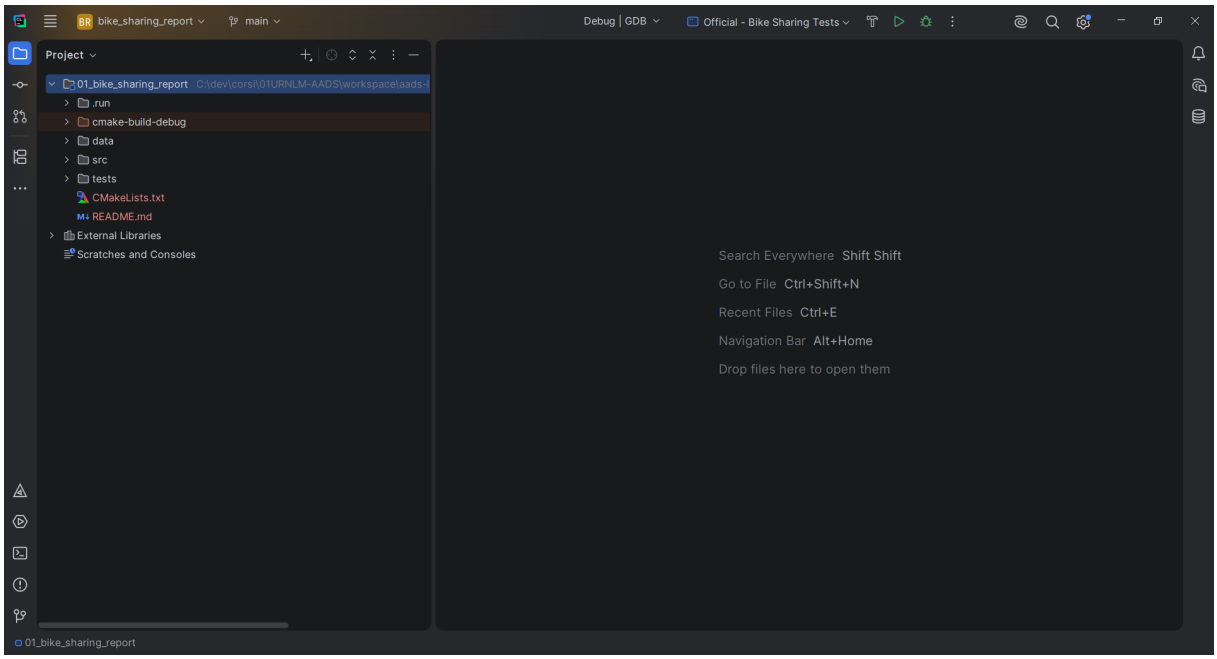


3. If CLion asks whether you trust the project, confirm that you do.
4. Wait while CLion detects `CMakeLists.txt` and loads the CMake project.

CLion may display a notification saying that the CMake project needs to be loaded or reloaded. Click **Load CMake Project** or **Reload CMake Project** and wait for the operation to finish before running anything.

If the notification is no longer visible, reload manually with **Tools | CMake | Reload CMake Project**. A reload updates CLion's knowledge of targets and project settings; it does not delete or overwrite your source code.

When loading has completed, the prepared course configurations appear in the Run configuration selector.



Work on the source files

Your implementation belongs under the exercise's `src/` directory. You may organize it in the way that best reflects your solution: source and header files can be placed directly in `src/` or grouped into meaningful subdirectories.

For example, both of these organizations are acceptable:

```
src/
├── main.c
├── queue.c
└── queue.h
```

```
src/
├── main.c
├── collections/
│   ├── queue.c
│   └── queue.h
└── input/
    ├── parser.c
    └── parser.h
```

The course project is expected to discover and compile every `.c` file under `src/`, including files in its subdirectories. Header files are included by source files but are not compiled on their own. Keep exactly one `main` function in the program.

After creating, deleting, moving, or renaming source files, reload the CMake project so CLion can update the target. You should not need to list filenames manually or modify `CMakeLists.txt`. If a `.c` file under `src/` is still not compiled after a reload, report the problem to the course staff.

Preserve supplied function prototypes

Some exercises provide a header containing function prototypes that you must implement. A supplied prototype is part of the exercise's public interface and must not be changed.

Preserve exactly:

- the function name;
- the return type;
- the number and order of parameters;
- every parameter type and qualifier.

The function definition in a `.c` file must match the supplied declaration. For example, if the exercise provides:

```c
int collect_items(const Item items[], int count, int indices[]);
```

implement that exact interface. Do not rename it, change its return type, swap parameters, or replace their types.

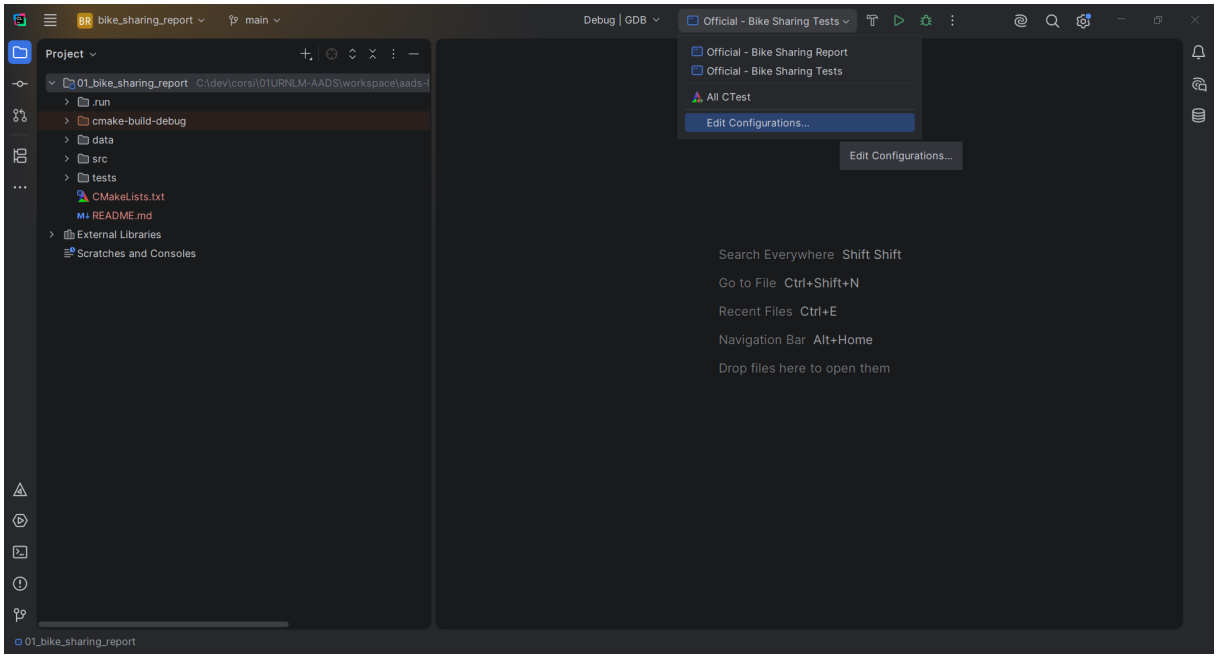
Automatic tests include the supplied headers and call these functions directly. If you change a prototype, the tests may no longer compile or may be unable to call your implementation at all.

You may always add private helper functions and new modules when they improve your solution. You may also add new declarations for your own functions. These additions must not alter or replace the prototypes already provided by the exercise.

Prepared and automatic Run configurations

CLion uses Run configurations to remember what should be executed and with which settings. Every prepared course configuration has a name beginning with:

`Official -`



Examples:

```
Official - Bike Sharing Report
Official - Bike Sharing Tests
```

CLion may also create configurations automatically from CMake targets. Their names are often lowercase and contain underscores, such as `bike_sharing_report`. These automatic configurations are not the prepared course configurations and may not contain the required input argument or working directory.

For the normal laboratory workflow, select the configuration beginning with **Official -**.

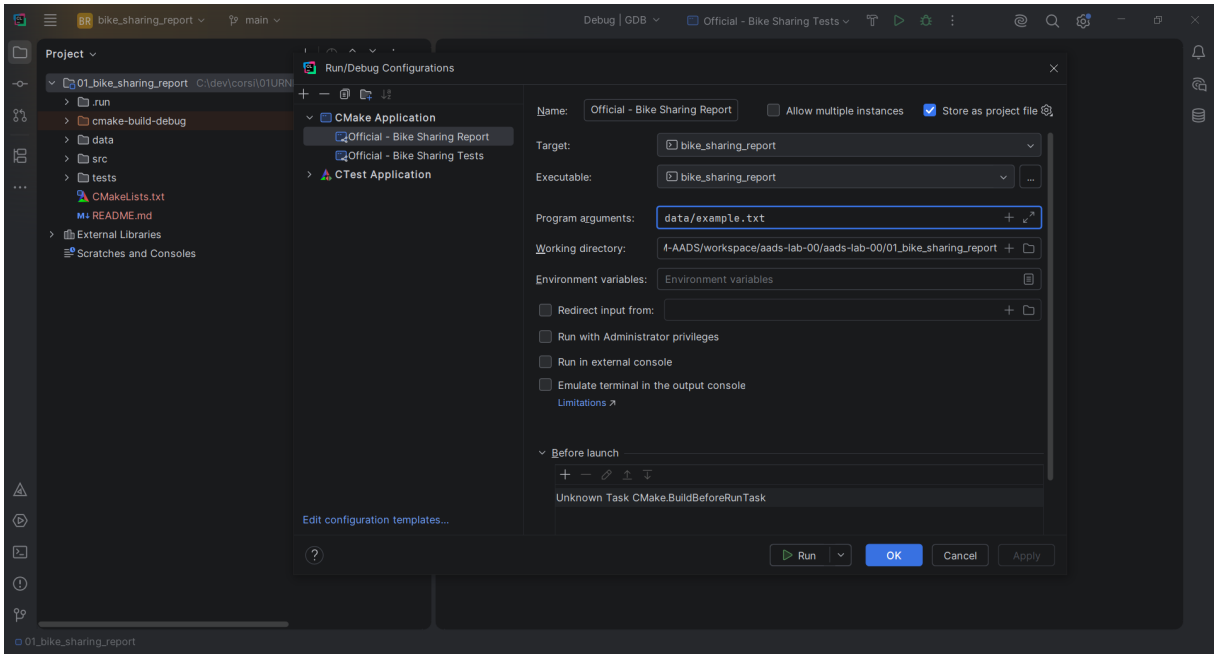
What a prepared configuration contains

A prepared program configuration defines:

- the executable target;
- the exercise directory as the **Working directory**;
- a default value under **Program arguments**, when the program needs one;
- **Build** under **Before launch**.

A prepared test configuration similarly selects the test executable and builds it before running it.

You should not edit the XML files under `.run/`. Use CLion's **Run | Edit Configurations** dialog when you need to inspect or change a setting locally.



Build automatically before Run, Debug, and Tests

You do not need to invoke **Build Project** manually before using an official configuration.

Whenever you click **Run**, **Debug**, or start the prepared tests, CLion first executes the **Build** step listed under **Before launch**. The first build compiles the complete target. Later builds are incremental and normally compile only source files that changed, so the program always uses the latest saved version of your code.

If CLion asks for a separate build instead of performing it automatically:

1. load or reload the CMake project;
2. select the configuration beginning with **Official -**;
3. open **Run | Edit Configurations**;
4. check that **Build** appears under **Before launch**.

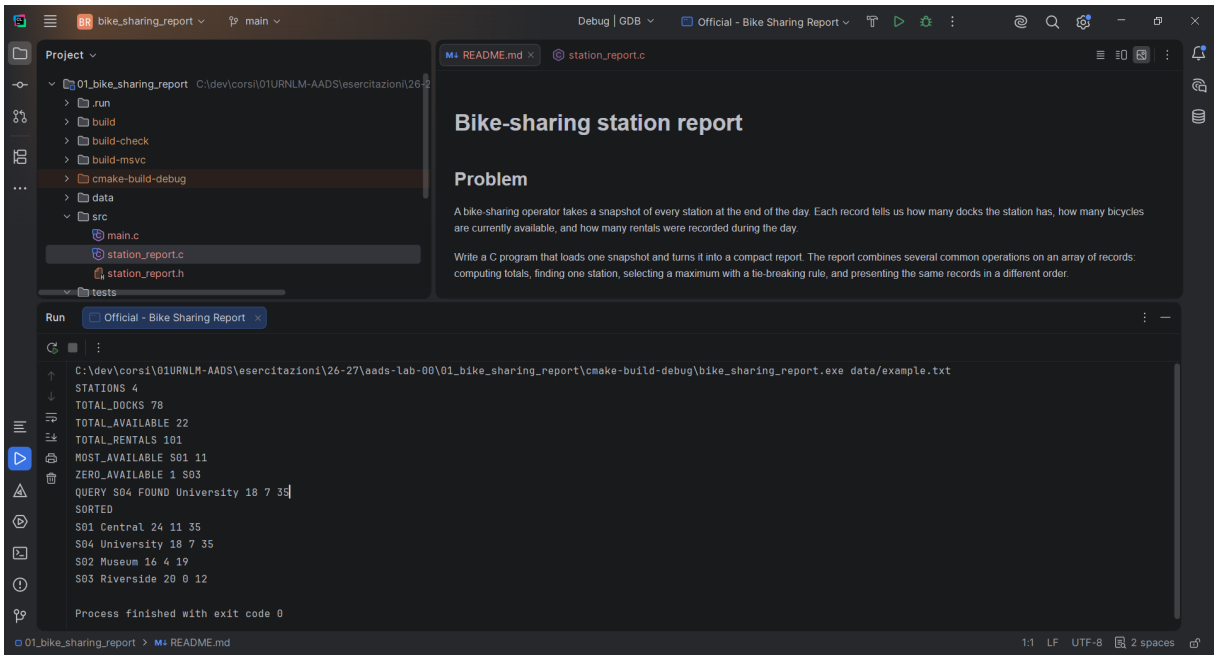
Do not remove the Build step, because otherwise CLion may execute an older binary.

Run a program

1. Read the exercise README and identify its input interface and supplied files.
2. Select the corresponding **Official - ...** program configuration.
3. Click **Run**.
4. Read the program output in the Run tool window.

When an exercise supplies `data/example.txt`, the prepared configuration normally already passes that path as a program argument.

Relative input paths are resolved from the exercise project directory. This is why the prepared **Working directory** must remain the directory containing `CMakeLists.txt`.



Input files supplied by an exercise

Normal example inputs are stored under the exercise's `data/` directory. The exercise README lists the available files and explains the case represented by each one.

Typical paths are:

```
data/example.txt
data/empty_input.txt
data/duplicates.txt
```

Keep distributed files in their original locations. Do not copy them into `src/`, `.idea/`, or a generated build directory.

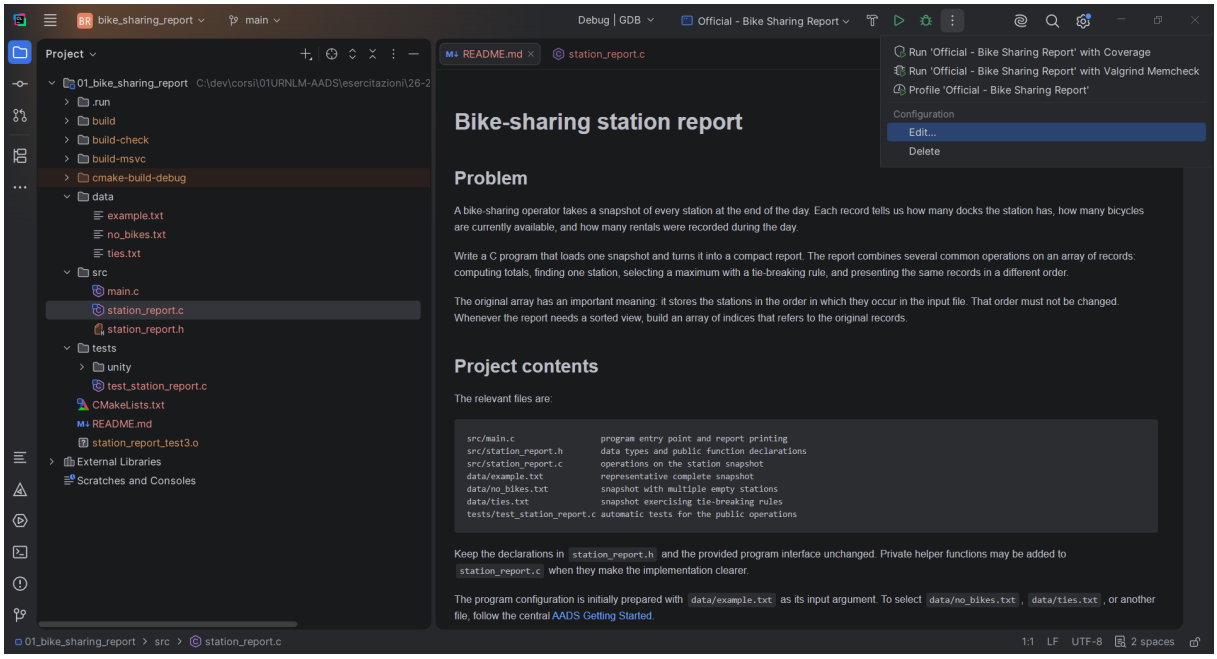
Change the input passed by a Run configuration

When a program accepts a file path, the prepared configuration normally starts with one input already loaded, often:

`data/example.txt`

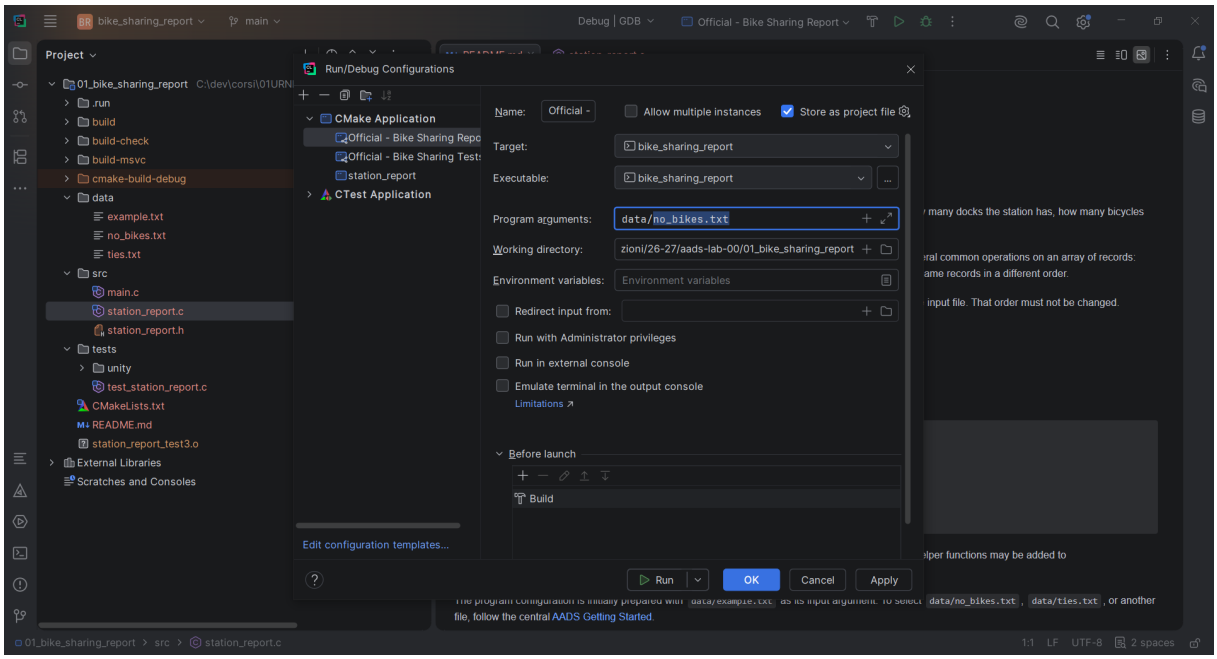
To run the same program with another file listed in the exercise README:

1. select the relevant **Official - ...** program configuration and edit it;



2. find the **Program arguments** field;
3. replace the existing path with the new relative path, for example:

   `data/no_bikes.txt`



4. leave **Working directory** set to the exercise project directory;
5. leave **Build** enabled under **Before launch**;
6. click **Apply**, then **Run**.

This changes only which data is passed to the program. It does not require any change to the C source code or `CMakeLists.txt`.

Preserve the default configuration

If you want to keep the distributed example readily available, duplicate the official configuration before changing it:

1. select the official configuration in **Run | Edit Configurations**;
2. choose **Duplicate Configuration**;
3. give the copy a descriptive name, such as `Bike Sharing - no bikes`;
4. change **Program arguments** in the copy;
5. apply and run the copy.

Duplicating is recommended when you expect to alternate between several input files. Directly editing the official configuration is also acceptable for a quick local experiment; restore the documented default path when you want to return to the original setup.

Paths and multiple arguments

Use paths relative to the exercise directory rather than absolute paths tied to your computer. If a filename contains spaces, quote the complete path.

Some programs accept more than one argument. In that case, preserve their documented order. For example:

```
min data/min_example.txt
```

The exercise README will define what each argument means.

Debug a program

Use the same official program configuration for debugging:

1. set a breakpoint by clicking in the gutter beside a source line;
2. select the official program configuration;
3. click **Debug** instead of Run;
4. step into or over functions and inspect variables, the call stack, pointers, and memory as needed.

The configured input arguments and automatic Build step are also used during Debug.

Run automatic tests

Some exercises provide automatic tests.

A test is a small program that calls part of your implementation with controlled input and compares the result with the expected value. Tests are particularly useful when an exercise provides function prototypes: each public operation can be checked separately, without having to diagnose the complete program all at once.

Run the tests frequently, for example after implementing or correcting one function. This gives you quick feedback and usually makes it easier to identify which operation introduced a problem.

When a test configuration is provided:

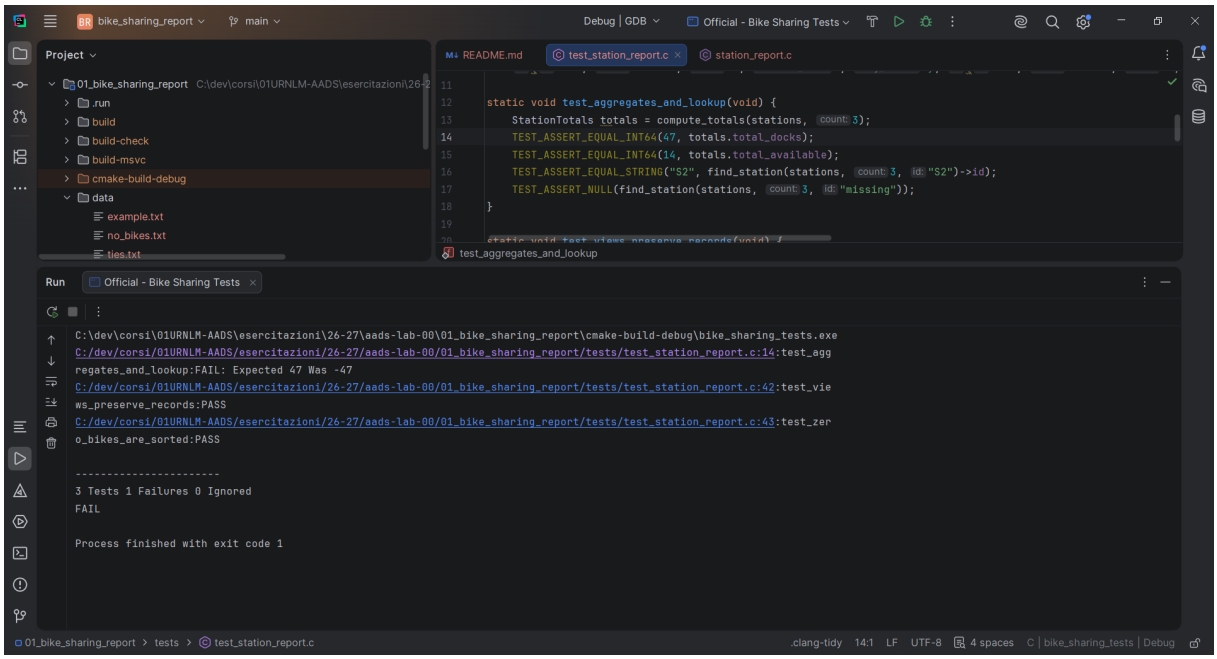
1. select the **Official - ... Tests** configuration;
2. click **Run**;
3. read the complete result in the Run tool window.

The configuration builds the latest code before starting the tests.

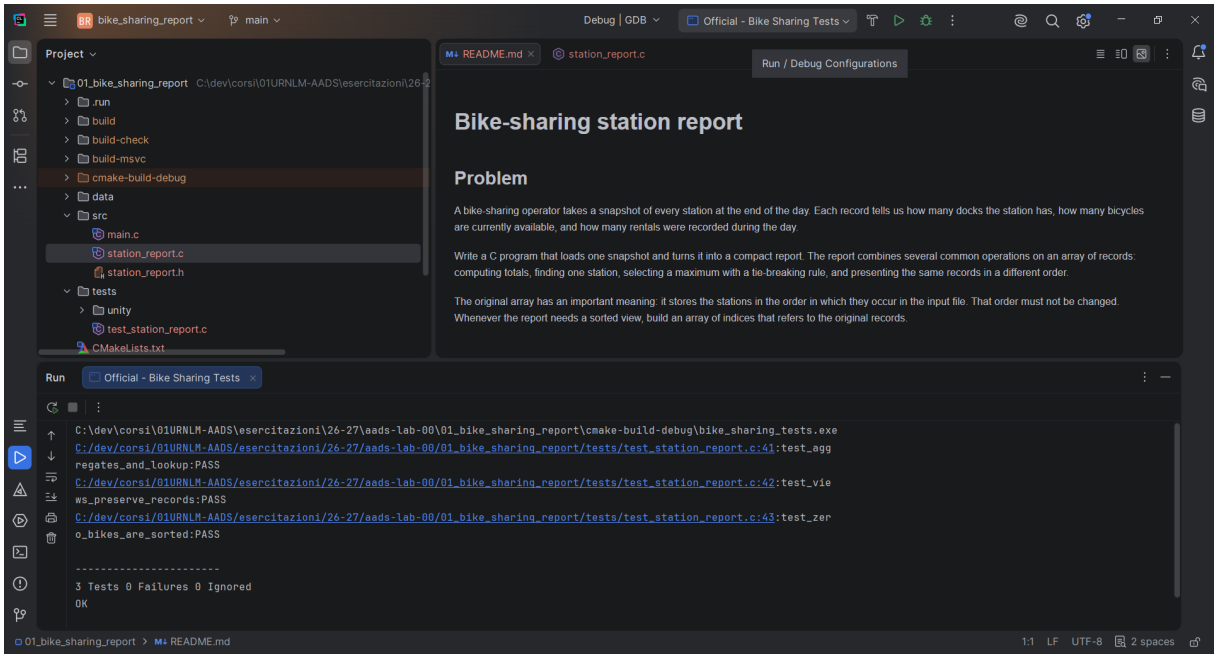
When a test fails, its output normally identifies:

- the name of the failing test;
- the source line containing the failed check;
- the expected value;
- the value produced by your implementation.

In the following deliberately failing example, two tests pass while one test reports a value different from the expected result:



Use this information to inspect and correct your implementation, then run the suite again. A successful execution reports that every supplied test passed:



Do not modify the provided tests, their data, or the supplied function prototypes merely to make a failure disappear. The purpose of a test is to reveal a discrepancy in the implementation, not to redefine the expected behavior.

Passing all supplied tests is an important check, but it does not prove that the complete program is correct for every possible input. The exercise README remains the authoritative specification. Also run the complete program with the documented input files and consider relevant boundary cases.

Not every exercise includes automatic tests. When none are provided, use the documented input files

and output contract for manual verification.

Files that should normally not be modified

Unless an exercise explicitly says otherwise, **do not modify**:

```
CMakeLists.txt
.run/
tests/
```

Do not modify supplied public function prototypes. Add your implementation, private helpers, and any new modules under `src/` as explained above.

If an exercise does not open or run correctly

Before reporting a problem, verify that:

- CLion is version 2026.2 or newer;
- the complete ZIP archive was extracted;
- you opened the individual exercise directory;
- the open directory directly contains `CMakeLists.txt`;
- CMake finished loading or reloading without errors;
- the selected configuration begins with **Official -**;
- its target exists and **Build** appears under **Before launch**;
- the documented input path exists under `data/`;
- **Working directory** is the individual exercise directory;
- `CMakeLists.txt` and `.run/` were not removed or manually edited.

Useful links

- CLion download: https://www.jetbrains.com/clion/download/
- CLion documentation: https://www.jetbrains.com/help/clion/
- CLion debugging: https://www.jetbrains.com/help/clion/debugging-code.html
- CMake project reload: https://www.jetbrains.com/help/clion/reloading-project.html